MNXB11

Project Report

Authors
Macaila Schafer
Matei Popescu
Lily-Jeanne Clement

November 3, 2026

1 Data Cleaning and Extraction

The original data files contained both the data and information in text regarding the data. To clean the files into a readable csv file without the text, we used a .sh cleaning script written by Florido Paganelli. This script develops different versions of the data file but the one that we preferred was a script with each line having the format of "year-month-day hour:minute:second temperature quality". The relevant parts of the cleaner script for this data format is shown.

To start, using the script above, all the beginning lines with just text are completely removed. This works by reading the first lines of actual data with "STARTLINE" and directs the data to an output data file with ">".

```
echo "Removing the first $STARTLINE lines, result in clean1_${DATAFILE}"
tail -n +$STARTLINE original_${DATAFILE} > clean1_${DATAFILE}
```

Then it cuts the text that is written next to data with the code below by using "cut" to select only the first 5 columns and redirects that to an output data file:

```
echo "Selecting only relevant columns, result in clean2_${DATAFILE}"
cut -d';' -f 1,2,3,4,5 clean1_${DATAFILE} > clean2_${DATAFILE}
```

Lastly, it removed the semicolons that separate the data with "sed" and the semicolons were replaced with spaces and created the final cleaned csv file. The substitution is given with "'s/;/ /g'":

```
echo "Substituting the ; with spaces, result in baredata_${DATAFILE}"
sed 's/;/ /g' clean2_${DATAFILE} > baredata_${DATAFILE}
```

The final version of the cleaned data file is of the desired format:

```
1780-01-01 07:00:00 -0.5 Y
1780-01-01 13:00:00 1.0 Y
1780-01-01 21:00:00 0.8 Y
1780-01-02 21:00:00 -3.0 Y
1780-01-03 13:00:00 1.3 Y
1780-01-03 21:00:00 -6.8 Y
1780-01-04 13:00:00 -9.0 Y
1780-01-05 13:00:00 -7.0 Y
1780-01-05 21:00:00 -10.0 Y
1780-01-06 21:00:00 -14.0 Y
```

Once we had the cleaned data file, the plan was to create a ROOT file with a branch for the variables: year, month, day, hour, minute, second, temp, and quality. This was done through a Measurement.cxx file and a DataExtraction.cxx file. The outlines for each script was used from example scripts by Oxana Smirnova and Einar Elen.

The first script, Measurement.cxx was used to define how a measurement is accessed and stored:

```
//Measurement.cxx
#include "Measurement.h"
```

```cpp
//Constructor to initialize variables
Measurement::Measurement(int year, int month, int day, int hour, int minute, int second, double temp
    : year{year}, month{month}, day{day}, hour{hour}, minute{minute}, second{second}, temp{temp}, qu

}

//getter functions to get the value of an object and be stored in the member value (the returned var

int Measurement::get_year() const{
    return year;
}

int Measurement::get_month() const{
    return month;
}

int Measurement::get_day() const{
    return day;
}

int Measurement::get_hour() const{
    return hour;
}

int Measurement::get_minute() const{
    return minute;
}

int Measurement::get_second() const{
    return second;
}

double Measurement::get_temp() const{
    return temp;
}

char Measurement::get_quality() const{
    return quality;
}


bool Measurement::is_valid() const{
    return true;
}
```

The DataExtraction.cxx code is able to then read the csv file and use Measurement.cxx for the

objects defined:

```
//DataExtraction.cxx
#include "DataExtraction.h"


//Measurment is a class defined in Measurement.h
//define the function read_measurements with the name given by the filename parameter
std::vector<Measurement> read_measurements(const std::string& filename) {
  //std::ifstream is used to read the .csv file and create an input file stream named file
  //where {filename} initializes the stream
  std::ifstream file{filename};
  //creates empty measurements vector where objects from Measurement will be stored
  std::vector<Measurement> measurements;


//define variables needed
  int year, month, day;
  int hour, minute, second;
  double temp;
  char quality;

  char dash;
  char colon;

  //Extract values from the file stream in a while loop
  // example: file >> variable will extract data from file and store the data in variable
  // In this case it expects to read: year-month-day hour:minute:second temp quality
  while (file >> year >> dash >> month >> dash >> day
    >> hour >> colon >> minute >> colon >> second >> temp >> quality) {
      //create object measurement which is used to test if the measurement is valid
      //using the .is_valid member function that returns true if it is valid
    Measurement measurement{year, month, day, hour, minute, second, temp, quality};
    if (measurement.is_valid()) {
      //.push_back member function is to add to the vector with evey iteration
      measurements.push_back(measurement);
    } else {
      std::cerr << "Warning: Skipping invalid measurement: ("
                << year << "-" << month << "-" << day << ","
                << hour << ":" << minute << ":" << second << ","
                << temp << "," << quality << ")"
                << std::endl;
                //std::endl is a stream manipulator that inserts a new line
                //(used in place of /n)
                //and flushes the output stream
                //(meaning it forces data in a memory buffer to move)
    }
  }
```

```
  return measurements;
}

//save the measurements to a ROOT file
//having the output file pass into the function allows to reuse this script
//with different output files
void persist_measurements(const std::vector<Measurement>& measurements,
                          const std::string& output_filename) {

  //create ROOT Tfile
  //c_str() is a function that returns a pointer (similar to using: const char*)
  TFile output_file{output_filename.c_str(), "RECREATE"};
  //create TTree with the name "MNXB11" and create datasets with a branch for each variable
  TTree tree{"MNXB11", "MNXB11"};
  Double_t temp;
  Int_t year;
  Int_t month;
  Int_t day;
  Int_t hour;
  Int_t minute;
  Int_t second;
  Int_t quality;
  tree.Branch("temp", &temp);
  tree.Branch("year", &year);
  tree.Branch("month", &month);
  tree.Branch("day", &day);
  tree.Branch("hour", &hour);
  tree.Branch("minute", &minute);
  tree.Branch("second", &second);
  tree.Branch("quality", &quality);

//auto& is used to make a reference whos data type is determined by the compiler
  for (const auto& measurement : measurements) {
    year = measurement.get_year();
    month = measurement.get_month();
    day = measurement.get_day();
    hour = measurement.get_hour();
    minute = measurement.get_minute();
    second = measurement.get_second();
    temp = measurement.get_temp();
    quality = measurement.get_quality();

    tree.Fill();
  }
  tree.Write();

}
```

Within the DataExtraction.cxx file, there are notes to explain each portion of the script. As an overview, it starts by using the std::ifstream class to read files and create a vector using std::vector. The while loop extracts the data from the data file in the expected format and order that is shown in the parameters for the while loop. Using the .push_back member function it adds a Measurement object to the empty vector. Persist_measurements is created to save the measurements to an output file. The ROOT file is created using the output file and "RECREATE" with a tree named "MNXB11". Each variable is then connected to its own branch in the ROOT tree by defining them as ROOT variables and connecting them each to a branch using tree.Branch(). The for loop at the end is used to get the stored values using the getter functions defined in Measurement.cxx. The values are stored with tree.Fill() and the tree.Write writes the tree to the file.

2 Data Analysis

This part of the report discusses the methodology and general coding practices used in the assignment repository. First, we discuss the common analysis methods used by the several different subtasks and then summarise the specific approach used in each.

2.1 Common Implementation and Shared Methods

The analysis reads the cleaned input data from `measurements_lund.root` that stores a `TTree` named `MNXB11`. In `ReadData.cxx`, ROOT binds `year`, `month` and `day` to `Int_t` variables and `temp` to a `Double_t`. Only these branches are enabled. Invalid results from the reading methods cause the code to stop and throw exceptions.

Daily means. For year y and calendar date d, let $n_{y,d}$ be the number of supplied readings and $T_{y,d,i}$ the ith temperature. The quantity used throughout is

$$D_{y,d} = \frac{1}{n_{y,d}} \sum_{i=1}^{n_{y,d}} T_{y,d,i}, \qquad n_{y,d} > 0.$$

`DailyTemperature` stores only a sum and count. The sum uses `long double` to reduce rounding during accumulation, and the returned mean is `double` for the sake of convenience. From `AnalysisCommon.cxx`:

```
void DailyTemperature::add(double temperature) {
  sum += temperature;
  ++count;
}
double DailyTemperature::mean() const {
  // Precondition: count > 0; analysis functions check this before calling.
  return static_cast<double>(sum / count);
}
```

Each observed date subsequently receives equal weight, regardless of its reading count. This avoids giving a year greater influence merely because it has more readings per day.

Calendar and missing observations. `TemperatureData` maps each observed year to a vector of 367 daily accumulation variables. Index 0 is unused, while indices 1–366 follow a common leap

calendar: January 1 is 1, February 29 is 60, March 1 is 61 and July 1 is 183. Non-leap years leave slot 60 empty, so matching month/day labels remain aligned without issues with data formatting. We decided it was more reasonable to skip empty dates if they occurred.

Common statistics. For any non-empty vector $x_1, \ldots, x_N$, `summarize()` calculates

$$\bar{x} = \frac{1}{N}\sum_i x_i, \qquad s = \sqrt{\frac{1}{N}\sum_i (x_i - \bar{x})^2}.$$

The two-pass calculation first obtains $\bar{x}$ and then sums squared deviations. The denominator N describes the available population; this is **not**, technically, the standard error $s/\sqrt{N}$! An empty vector returns count zero, which is interpreted as data being unavailable. A 1-dimensional vector simply has $s = 0$ by definition. We chose this methodology to make our statistical analysis without necessarily assuming a Gaussian distribution from the very get-go.

2.2 Subtask 3.1

The first question our implementation needs to address is to calculate the daily mean and standard deviation. `analyzeGivenDay()` in `GivenDay.cxx` collects one $D_{y,d}$ for each year containing date d. The current configuration in `main.cxx` chooses August 23. With $Y_d = \{y : n_{y,d} > 0\}$ and $N_d = |Y_d|$, the answers are

$$\mu_d = \frac{1}{N_d}\sum_{y \in Y_d} D_{y,d}, \qquad \sigma_d = \sqrt{\frac{1}{N_d}\sum_{y \in Y_d} (D_{y,d} - \mu_d)^2}.$$

σ_d therefore measures variation *between years on this date*, rather than fluctuations in temperature on a given day or some form of measurement uncertainty.

```
std::vector<double> temperatures;
for (const auto& year : data) {
  if (year.second[slot].count > 0) {
    temperatures.push_back(year.second[slot].mean());
  }
}
```

`slot` is the selected date's calendar index; `year.second` is that year's vector.

Calling `summarize(temperatures)` then supplies the count, mean and SD. Invalid dates and non-finite results are simply unused. An unobserved date yields no statistical result and has no weight, to avoid bias in that sense. Our implementation estimates an interval frequency for a given temperature value, so directly computing the probability of observing one specific temperature is not possible, strictly speaking. For configured bounds $a < b$,

$$\widehat{p}_d([a, b)) = \frac{\#\{y \in Y_d : a \le D_{y,d} < b\}}{N_d}.$$

The current interval is $[15, 20)$: 15 is included, and 20 is excluded. The relevant counting code is:

```
int inInterval = 0;
for (int i = 0; i < summary.count; ++i) {
  if (temperatures[i] >= probabilityLow &&
      temperatures[i] < probabilityHigh) {
    ++inInterval;
  }
```

```
}
```

Division uses `static_cast<double>(inInterval) / summary.count`, avoiding integer truncation.

2.3 Subtask 3.2

Up to the statistical analysis, subtask 3.2 is an application of subtask 3.1 to every single day in the calendar. We are therefore comparing day mean variances. We utilise the exact same collection and `summarize()` operations for slots 1–366. It reports $(d, N_d, \mu_d, \sigma_d)$, with N_d recalculated separately for each date. Thus, missing observations do not interfere. February 29 uses only years with observations on February 29. Similarly to the above subsection, the supplied uncertainty magnitude is σ_d, the year-to-year descriptive SD defined above. It is not a confidence interval for μ_d, or a measurement error.

2.4 Subtask 3.3

For this task, we must analyse year extremes. `analyzeYearExtremes()` in `YearExtremes.cxx` searches only the observed daily means of each year:

$$M_y = \max_{d:n_{y,d}>0} D_{y,d}, \qquad m_y = \min_{d:n_{y,d}>0} D_{y,d}.$$

It retains every date attaining each extreme. These are extrema of *daily means*, not extrema of individual temperature readings. The maximum search is below; a separate, analogous comparison finds the minimum.

```
    double temperature = year.second[slot].mean();
    if (warmDates.empty() || temperature > maximum) {
      maximum = temperature;
      warmDates.clear();
      warmDates.push_back(slot);
    } else if (temperature == maximum) {
      warmDates.push_back(slot);
    }
```

The surrounding loop skips empty dates before calling `mean()`. Although `maximum` and `minimum` start at zero, the empty-date-list tests accept the first observation unconditionally.

Equal influence for tied years. If year y has k_y tied warmest dates, each receives weight $1/k_y$. Writing their set as W_y, the accumulated date weight is

$$H_W(d) = \sum_y \frac{\mathbf{1}_{\{d \in W_y\}}}{|W_y|}, \qquad \sum_d H_W(d) = N_{\text{years}}.$$

The same construction applies to coldest dates. ROOT `TH1D` objects act as double-valued accumulator variables, so they conveniently allow fractional weights:

```
  double warmWeight = 1.0 / warmDates.size();
  double coldWeight = 1.0 / coldDates.size();
  for (int i = 0; i < static_cast<int>(warmDates.size()); ++i) {
    warm->Fill(warmDates[i], warmWeight);
  }
```

This prevents years with several tied dates from being overcounted and biasing the resulting mean.

Likeliest extreme. `printModes()` returns the date (or dates) with maximum accumulated weights and reports $\max_d H_W(d)/N_{\text{years}}$, or its cold counterpart. A tolerance of 10^{-9} compares bin weights.

Year boundary handling. Calendar dates are cyclic: December 31 and January 1 are neighbours. Instead of fitting a wrapped Gaussian to the extreme-date distribution, we also calculate

$$d' = (d - 183 + 366) \bmod 366.$$

July 1 becomes 0, December 31 becomes 183, and January 1 becomes 184.

2.5 Task 3.4

In this subtask, we compare the temperature variation in Lund in 1990 and 2000. `compareYears()` in `CompareYears.cxx` requires the requested years to exist and the earlier year to precede the later one. It reports each year's mean and standard deviation over its own observed dates. To compare equal calendar coverage, it additionally forms the shared-date set S and differences

$$\Delta_d = D_{2000,d} - D_{1900,d}, \quad d \in S; \qquad \overline{\Delta} = \frac{1}{|S|} \sum_{d \in S} \Delta_d.$$

The pairing code executes the following statements only inside `if (hasEarly && hasLate)`, excluding dates absent from either year:

```
double first = earlier->second[slot].mean();
double second = later->second[slot].mean();
double difference = second - first;
pairedEarly.push_back(first);
pairedLate.push_back(second);
differences.push_back(difference);
monthlyDifferences[month - 1].push_back(difference);
```

Positive differences mean warmer daily means in 2000. February 29 contributes to 2000's own-year summary but not the paired comparison, since 1900 is not a leap year. `summarize(differences)` supplies $\overline{\Delta}$ and its descriptive deviation s.

3 Final Figures

The analyzed data was plotted in four different graphs using ROOT libraries. Two functions were set up to control the general formatting of graphs : GraphStyle1 for those with one canvas per image, and GraphStyle2 for those with two. Graph specific formatting was done within each analysis file.

GraphStyle1 and GraphStyle2 were used to set common margins, title and labels sizes, offsets, as well as centering. Graphstyle1 was overloaded to accommodate both TH1D and TGraphErrors, and GraphStyle2 accommodates both TH1D and THGraph classes.

3.1 Subtask 3.1

Figure 1, compiles the mean daily temperature on August 23 throughout the years in Lund. The mean temperature is 16.41°C , and the standard deviation is 2.49 °C .

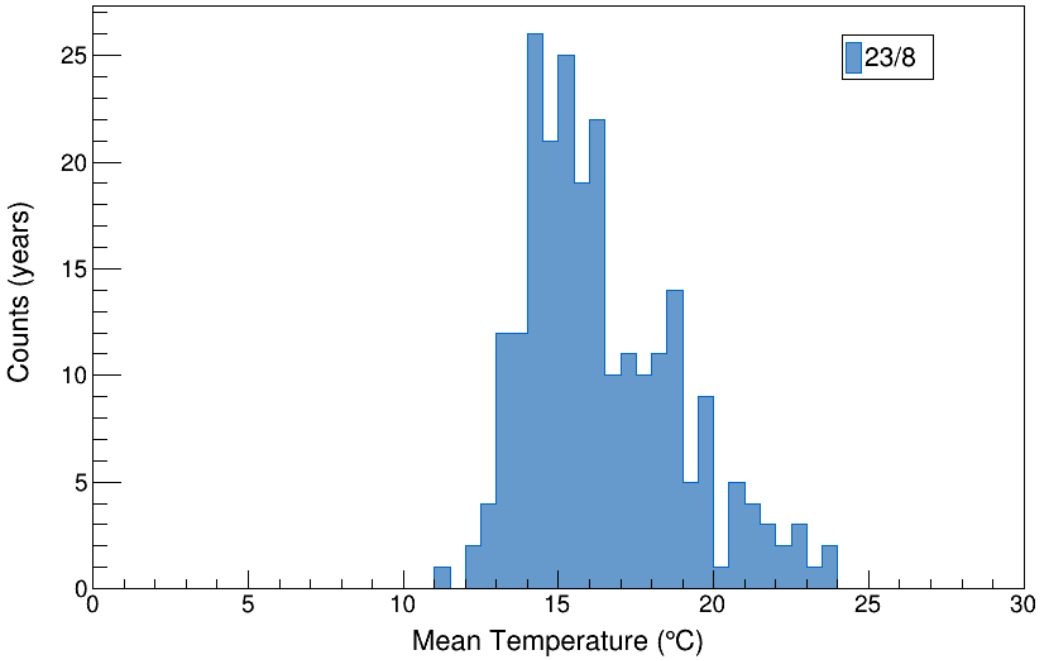


Figure 1: Histogram of the mean daily temperature on August 23 in Lund.

3.2 Subtask 3.2

Figure 2 shows the mean temperature and standard deviation for everyday of the year in Lund. As expected, we observe higher temperatures (around 17°C) in the summer months, and colder temperatures (around 0°C) in the winter months. The standard deviation is slightly more important in the winter than in the summer.

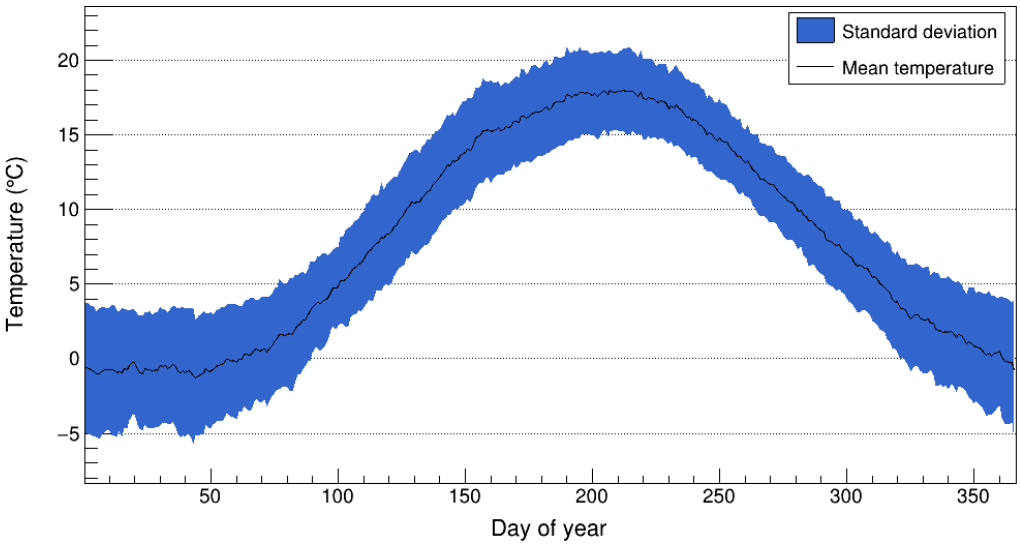


Figure 2: Mean daily temperature throughout the year in Lund. The dark line represents the mean temperature. The colored area represents the standard deviation.

3.3 Subtask 3.3

Figure 3 shows which days of the year were the coldest or warmest at some point in Lund. The graph was separated into two canvas. The top one shows the year spanning from January 1 to December 31. The bottom one focuses on the coldest days and goes from July 1 to June 31, centering on winter and showing the winter trends without any cut.

As expected, the warmest day of the year tends to be in summer, with July 12 being the most common, and the coldest day of the year tends to be in winter, with December 31 being the most common. It can be pointed out that some of the coldest days of the year seem to be in winter or very early spring. This seems to come from the data for a couple of years being incomplete and only having winter temperatures.

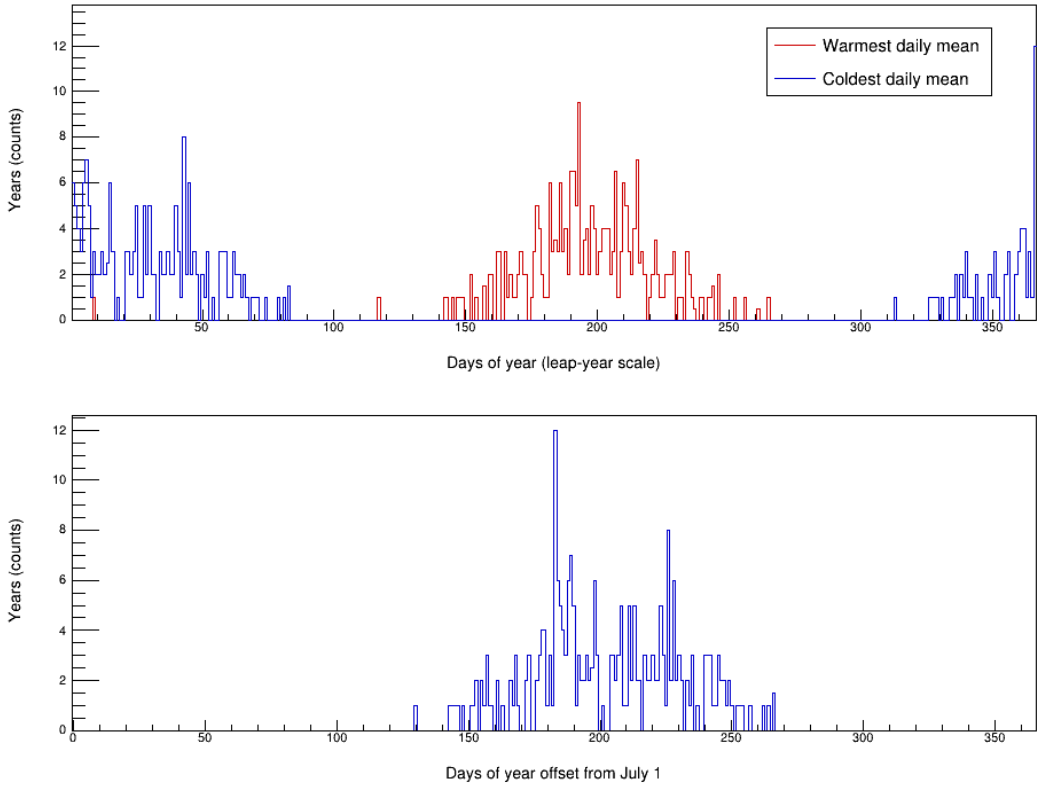


Figure 3: Histogram showing how many times each day has been the coldest (blue) or warmest (red) day of the year. Top : Calendar year from January 1 to December 31. Bottom : Year from July 1 to June 31. Only coldest days are shown here.

3.4 Subtask 3.4

Figure 4 shows the difference in daily temperature 100 years apart. The data was fitted by two sinusoidal functions in the top canvas. In the bottom canvas, the daily difference was fitted by a sinusoidal function, and its mean was fitted by a linear function.

Compared to 1900, 2000 had overall hotter winters and colder summers. The overall yearly mean temperature increased by 1.5°C, which corresponds to expected values from climate change.

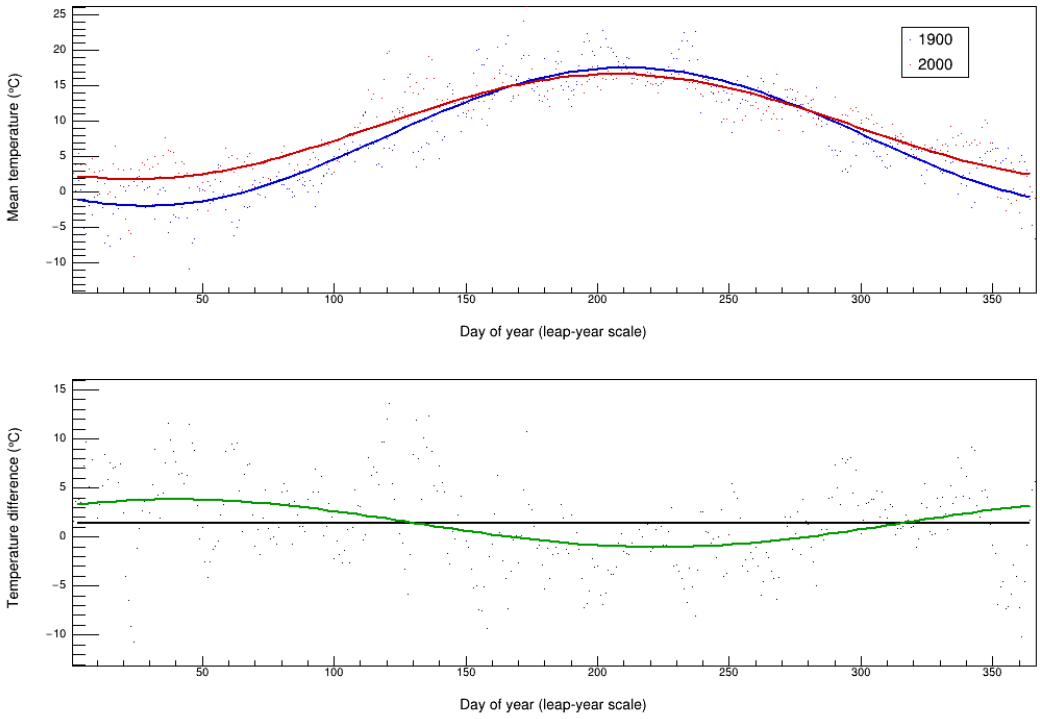


Figure 4: Temperatures in Lund in 1900 vs 2000. Top : Mean daily temperatures fitted with sinusoidal curves. Year 1900 in blue and year 2000 in red. Bottom : Daily temperature difference fitted with sinusoidal curve (green curve). The black line represents the mean yearly difference.